

TASK 3: ARCHITECTURAL DESIGN – CLOUD-BASED BUS PASS SYSTEM

Document Type: Cloud Systems Architecture & Deployment Guide

Project Track: Cloud Computing Internship

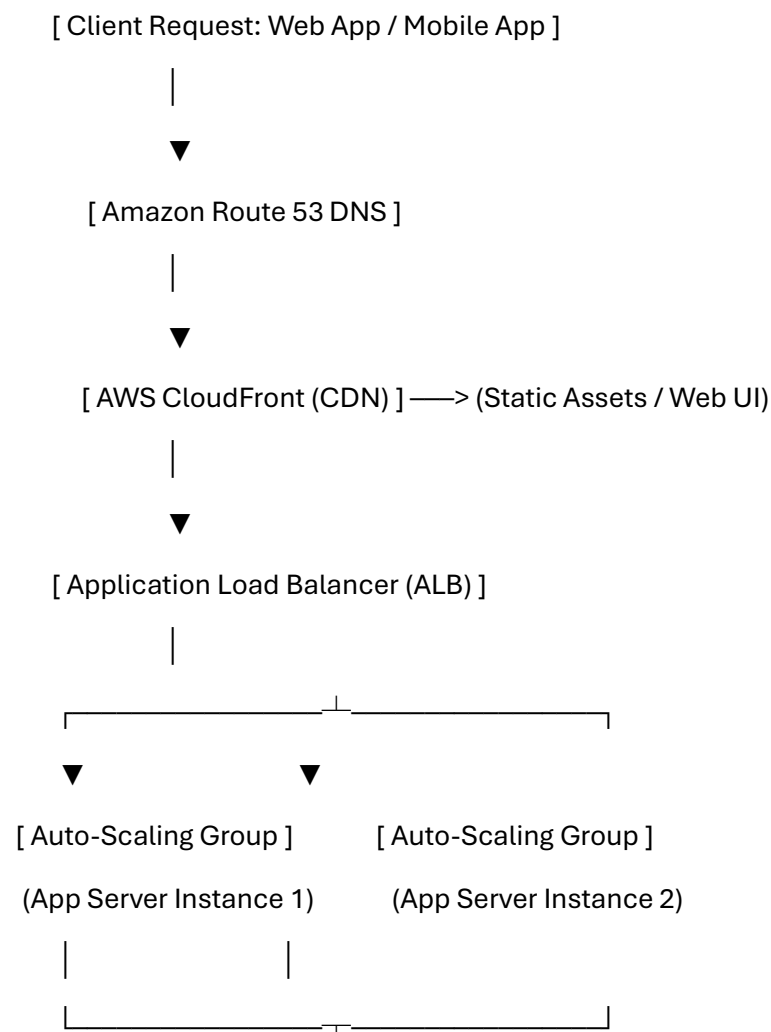
Prepared For: CodeAlpha Evaluation Board

1. System Overview & Core Objectives

The **Cloud-Based Bus Pass System** is a highly scalable, multi-tier web application designed to handle digital ticket sales, commuter pass generation, and real-time scanning validation. By leveraging a cloud-native architecture, this system completely mitigates the risks of traditional ticket platforms, such as physical data loss, data corruption, ticket fraud, and runtime site crashes caused by unexpected high-volume rush hours.

2. Global Cloud Architecture Map

To achieve high reliability and zero downtime, the system relies on a decoupled microservices blueprint distributed across public cloud infrastructure.





[Redis Cache Cluster] (Active Session States)



[Amazon Aurora PostgreSQL DB] (Primary / Replica)

3. Handling High Traffic & Dynamic Scalability

Traditional ticketing infrastructures crash under high load because they rely on fixed hardware. This cloud system addresses high concurrency using three core patterns:

Horizontal Auto Scaling

Instead of vertically modifying a single instance size, the compute layer uses virtual machines grouped in an **Auto-Scaling Group (ASG)** managed by a health coordinator.

- **Scale-Up Trigger:** If global CPU utilization crosses **70%** for more than 3 consecutive minutes, the scaling engine automatically provisions additional server nodes.
- **Scale-Down Trigger:** When passenger booking traffic drops (eg, during night hours), the engine gracefully removes redundant nodes to save infrastructure billing costs.

Load Balancing Lifecycle

An **Application Load Balancer (ALB)** acts as the single point of entry for user requests. It tracks downstream backend instances and dynamically routes incoming HTTP/HTTPS validation requests evenly using a **Round-Robin routing algorithm**, ensuring no single compute node bears a disproportionate transactional load.

4. Ticket Security & Loss Mitigation Framework

To fulfill the core requirement of preventing **ticket loss, fraud, and incorrect pricing**, the system uses an event-driven database synchronization approach:

- **Cryptographic QR Pass Generation:** When a commuter buys a bus pass, the system generates an encrypted payload signed with a unique hashing token string containing [UserID + PassID + ExpirationTimestamp]. This string is rendered as an immutable dynamic QR code on the user's mobile display, eliminating physical loss and ticket duplication counterfeiting.
- **Distributed Database Layer (ACID Compliance):** The core transaction ledger is backed by a managed relational engine (**Amazon Aurora PostgreSQL**) operating in a Multi-Availability Zone layout. If a primary data center suffers a physical

outage, a replica instance is automatically promoted to primary in under 30 seconds with **zero data loss** , ensuring commuter records are completely safe.

- **Pricing Engine Isolation:** To prevent unauthorized or incorrect fare changes during runtime calculations, bus pass tariffs are managed by immutable data validation rules locked on the cloud database layer.